UniSync

Project Stage 1

EECS 3311: Software Design

York University

Samreen Ahmed - 221419155

October 9, 2026

1.1 Project Description

UniSync will be an AI-powered planning application that helps university students organize their courses, assignments, quizzes, exams, extracurricular activities, and personal commitments all in one place. The AI planning agent will analyze students' workload, availability and other weekly commitments to create a realistic weekly schedule that prioritizes tasks, detest conflict, and actively adapts when changes are made. The application will also allow students to track their habits and goals to help them stay productive and improve overtime. The aim of UniSync is to provide aid to students and help them navigate their education and extracurricular activities in a simple and burden-free way.

The primary target users of Unisync are university and college students who manage multiple academic responsibilities alongside extracurricular activities, jobs, or personal commitments. The application is catered towards those students who struggle with prioritizing tasks, estimating the time required for assignments, and balancing their everyday lives.

UniSync’s AI will be able to:

- Retrieve information about the student's courses, tasks, deadlines, and commitments.
- Analyze upcoming deadlines, estimated task durations, and available time.
- Help prioritize tasks based on urgency and importance.
- Generate a personalized weekly schedule.
- Identify scheduling conflicts and periods of excessive workload.
- Suggest or perform schedule adjustments when new commitments arise.
- Use previous planning information and student preferences to help improve future plans.

An OpenAI GPT model will be used to build this application. For the initial design, a single LLM will be used to ensure the implementation remains manageable and focused.
UniSync will connect the user interface, AI agent, scheduling logic, and database. When a student makes a request, the AI agent will retrieve the relevant information and help create or update a plan. The scheduling logic will check for conflicts and ensure the plan is realistic. The proposed changes will then be shown to the student for approval before being saved.

1.2 Feature Specification

Feature 01 - Course Management

Students can add their courses and edit and view them through the “Courses” section of the GUI. The system will display it on the course list and will prompt users to input necessary information about courses such as course code, course name, and schedule. If required information is missing, duplicated, or invalid, the system displays an error and asks the student to correct it. AI involvement for this feature is deterministic.

Feature 02 - Academic Task and Deadline Management

Users can create, edit and track assignments, quizzes, exams and other tasks through the “Tasks” section of the GUI. Task name, associated course, deadline, estimated time for completion and priority will be fields that user has to input. If the required fields are missing, they will be notified and prompted to take actions. The AI involvement for this feature is deterministic.

Feature 03 - Extracurricular and Personal Commitment Management

Students can add, edit, and remove work shifts, extracurricular activities, appointments, and other personal commitments through the “Commitments” section of the GUI. They provide the activity name, date, start and end times, and whether it is recurring, which the system validates and adds to the calendar. If the times are invalid or overlap with existing commitments, the system alerts the student. The AI involvement for this feature is deterministic.

Feature 04 - Weekly Calendar and Timeline

Students can access the “Calendar” to view their classes, assignments, personal commitments, and planned study sessions in a weekly or monthly view. The system retrieves saved scheduling information and organizes it by date and time, allowing students to navigate between weeks and view or edit events in a visual way. If no events exist, an empty calendar is displayed with an option to add activities. The AI involvement for this feature is deterministic.

Feature 05 - AI Workload Analysis

Students can select “Analyze Workload” to evaluate their upcoming deadlines, estimated task durations, existing commitments, and available study time. The system will calculate their workload, and the AI agent will interpret the results to generate a summary highlighting urgent tasks, busy periods, and potential overload. The AI involvement for this feature is hybrid.

Feature 06 - AI Weekly Plan Generation

Students can select “Generate My Week” and enter scheduling preferences, such as preferred study hours and available time. The AI agent retrieves upcoming tasks, deadlines, and commitments, helps prioritize them, and works with the scheduling system to generate a personalized weekly plan. The system checks

for conflicts and displays the proposed schedule for student approval before saving it. The AI involvement for this feature is hybrid.

Feature 07- Schedule Conflict and Overload Detection
Students can view scheduling warnings through the “Calendar” or “Workload” section of the GUI. The system compares scheduled events, study sessions, deadlines, and available time to detect overlapping activities or excessive workloads, then displays warnings and AI-generated suggestions for resolving them. If no suitable solution exists, the system explains the conflict and asks the student to adjust their commitments or estimated durations for tasks. The AI involvement for this feature is hybrid.

Feature 08 - AI Adaptive Rescheduling
Students can request schedule adjustments through the “Calendar” when new commitments arise, tasks are delayed, estimated time of completion is miscalculated, or study sessions are missed. Using the current schedule, the agent proposes a revised plan that the system checks for conflicts. The updated schedule is displayed for approval before being saved, and if rescheduling is not possible, the agent suggests alternatives. The AI involvement for this feature is Hybrid.

Feature 09 - Progress and Habit Tracking
Students can use the “Progress” section to track completed assignments, study sessions, and personal habits by recording their completion status and dates. The system saves these records, updates progress statistics, and displays weekly summaries to help students monitor their goals. If records are missing or duplicated, the system handles them appropriately and displays accurate progress information. The AI involvement for this feature is deterministic.

Feature 10 - Natural-Language Planner Assistant
Students can communicate with the AI agent through a chat interface by entering questions or requests, such as asking about upcoming deadlines or requesting schedule changes. The agent interprets the message, retrieves relevant task and calendar information through application tools, and provides answers, recommendations, or proposed actions. If a request is unclear, the agent asks for clarification. The AI involvement for this feature is hybrid.

2.1 Class Diagram
The UML diagram is attached separately.

The patterns being used to implement UniSync are:
1. MVC Pattern:

UniSync will support both a GUI and a CLI. Without MVC, the code for displaying information and handling tasks, schedules, and AI features would be combined, making the application harder to organize and maintain. MVC is suitable because UniSync must support multiple interfaces that access the same functionality. Both the GUI and CLI can use the same controller without duplicating the underlying planning logic. However, changes to the GUI could affect the application logic, and the CLI might require a separate implementation of the same features. This would make the system harder to test.

Relevant classes for this pattern include:

- StudentFlowGUI - View: Displays courses, assignments, weekly schedules, and AI recommendations.
- StudentFlowCLI - View: Provides a text-based way for students to interact with the application.
- PlannerController - Controller: Receives requests from either interface and forwards them to the appropriate application components.
- StudentData, Course, AcademicTask, WeeklyPlan, and other data classes - Model: Represent and store the information used by the application.

2. Observer Pattern

When a student adds an assignment, changes availability, or approves a new weekly plan, the information displayed in the GUI should reflect the updated data. UniSync needs a way to notify necessary components when information changes without tightly coupling the data management logic to the GUI. The observer pattern allows the GUI to respond to these changes without requiring PlannerService to know the details of how the interface displays information.The application would need to manually refresh the GUI after every operation or repeatedly check whether data had changed. This could result in duplicated code, outdated displays, and stronger dependencies between the interface and application logic.

Relevant classes for this pattern include:

- ScheduleObserver - Observer interface: Defines a method for receiving notifications when planner information changes.
- StudentFlowGUI - Concrete observer: Implements the observer interface and refreshes its displayed information when notified.
- PlannerService - Subject: Maintains registered observers and notifies them after relevant changes.
- PlannerController - Coordinator: Processes user requests that may trigger updates through PlannerService.

3. Strategy Pattern

Every student has a certain preference when it comes to how their schedules are constructed. Some may want to prioritize assignments with the closest deadlines, while others may prefer distributing their workload more evenly throughout the week. Therefore, UniSync needs to be able to support different scheduling algorithms. The strategy pattern will allow different scheduling takes while maintaining the scheduling logic.

The classes involved in this pattern are:

- SchedulingStrategy - Strategy interface: Defines the common method for generating a weekly study schedule.
- DeadlineFirstStrategy - Concrete strategy: Prioritizes academic tasks based on their deadlines.
- BalancedWorkloadStrategy - Concrete strategy: Distributes study sessions more evenly across available days.
- WeeklyScheduler - Context: Uses the selected strategy to generate a weekly plan.

4. Adapter Pattern

UniSync will use a large language model (LLM) to interpret natural-language requests and assist with planning decisions. However, an external AI provider has its own API, request format, and response structure. The application needs a consistent way to communicate with the LLM without making the planning agent depend directly on a particular provider.

The classes involved in this pattern are:

- LLMClient - Target interface: Defines the operations StudentFlow expects from an LLM.
- OpenAIAdapter - Adapter: Implements LLMClient and translates requests into the format required by the OpenAI API.
- PlanningAgent - Client: Uses LLMClient to interpret student requests and generate planning recommendations.
- OpenAI API - Adaptee: The external service whose interface is being adapted.

2.3 Sequence Diagrams

Sequence diagram has been attached separately.

Task 3: Feature-to-Design Traceability

F	Description	Type	Use Case (UC)	Classes	Key Methods	Sequence Diagram	Design Patterns
1	Manage Courses	Deterministic	UC01	StudentFlowGUI, PlannerController, PlannerService, StudentData, Course	addCourse(), save()	SD01	MVC
2	Manage academic tasks	Deterministic	UC02	StudentFlowGUI, PlannerController, PlannerService, StudentData, AcademicTask	addTask(), save()	SD01	MVC
3	Manage Commitments	Deterministic	UC03	StudentFlowGUI, PlannerController, PlannerService, StudentData, Commitment	addCommitment(), save()	SD01	MVC
4	View weekly calendar	Deterministic	UC04	StudentFlowGUI, PlannerController, PlannerService, StudentData, WeeklyPlan	getCalendarAndWorkload(), getData()	SD02	MVC
5	Analyze workload	Deterministic	UC05	PlannerController, PlannerService, StudentData, AcademicTask	getCalendarAndWorkload(), calculateWorkload()	SD02	MVC
6	Generate weekly plan	Hybrid	UC06	PlannerController, PlannerService, WeeklyScheduler, SchedulingStrategy, ConflictDetector	generatePlan(), generate(), buildPlan(), hasConflict()	SD03	MVC, Strategy
7	Detect schedule conflicts	Deterministic	UC07	WeeklyScheduler, ConflictDetector, StudySession, StudentData	hasConflict()	SD03, SD04	Strategy
8	Reschedule weekly plan	Hybrid	UC08	PlannerController, PlannerService, WeeklyScheduler, SchedulingStrategy, ConflictDetector	reschedulePlan(), reschedule(), buildPlan(), hasConflict()	SD04	MVC, Strategy
9	Track habits and progress	Deterministic	UC09	StudentFlowGUI, PlannerController, PlannerService, StudentData, Habit	recordHabit(), record(), save(), update()	SD05	MVC, Observer
10	Use AI planner assistant	AI	UC10	PlannerController, PlanningAgent, AgentTools, PlannerService, LLMClient, OpenAIAdapter	ask(), respond(), getTasks(), getData(), generateResponse()	SD06	MVC, Adapter

Task 4: Explain How Each Feature Is Realized

F01 - Manage Courses

Related Use Case: UC01 - Manage Courses

Related Sequence Diagram: SD01 - Manage Courses, Tasks, and Commitments

Classes involved:

- StudentFlowGUI - allows students to enter and manage their courses.
- PlannerController - receives requests from the interface.
- PlannerService - handles the course management operation.
- StudentData - stores the student's course information.
- Course - represents an individual course.

Important methods: addCourse(), save()

Execution: When a student adds a course through the GUI, the request is sent to PlannerController, which passes it to PlannerService. The service adds the new Course to StudentData and saves the updated information. The GUI then shows the updated course list.

F02 - Manage Academic Tasks

Related Use Case: UC02 - Manage Academic Tasks

Related Sequence Diagram: SD01 - Manage Courses, Tasks, and Commitments

Classes involved:

- StudentFlowGUI - allows students to enter assignments, quizzes, and exams.
- PlannerController - receives task management requests.
- PlannerService - handles adding and updating tasks.
- AcademicTask - stores task details such as deadlines and completion status.
- StudentData - stores the student's academic tasks.

Important methods: addTask(), save()

Execution: The student enters an academic task, including its name, deadline, and associated course. PlannerController passes this information to PlannerService, which adds the task to StudentData. The updated information is saved so the student can view it later.

F03 - Manage Commitments

Related Use Case: UC03 - Manage Commitments

Related Sequence Diagram: SD01 - Manage Courses, Tasks, and Commitments

Classes involved:

- StudentFlowGUI - allows students to enter personal activities and extracurricular commitments.
- PlannerController - receives the request.
- PlannerService - handles commitment management.
- Commitment - represents an activity with a scheduled time.
- StudentData - stores the student's commitments.

Important methods: addCommitment(), save()

Execution: The student enters a commitment, such as a work shift, club meeting, or personal activity. The request goes through PlannerController to PlannerService, which adds the commitment to StudentData. The information is saved and can be considered when creating a weekly schedule.

F04 - View Weekly Calendar

Related Use Case: UC04 - View Weekly Calendar

Related Sequence Diagram: SD02 - View Calendar and Analyze Workload

Classes involved:

- StudentFlowGUI - displays the weekly calendar.
- PlannerController - requests calendar information.
- PlannerService - retrieves the relevant information.
- StudentData - contains the student's stored schedule information.
- WeeklyPlan - represents the planned study schedule.

Important methods: getCalendarAndWorkload(), getData()

Execution: When the student opens the weekly calendar, the GUI sends a request to PlannerController. The controller uses PlannerService to retrieve the student's courses, commitments, and study sessions. This information is returned to the GUI and displayed in a weekly calendar format.

F05 - Analyze Workload

Related Use Case: UC05 - Analyze Workload

Related Sequence Diagram: SD02 - View Calendar and Analyze Workload

Classes involved:

- PlannerController - receives the workload analysis request.
- PlannerService - coordinates the analysis.
- StudentData - provides the student's stored academic information.
- AcademicTask - contains deadlines and task details.

Important methods: getCalendarAndWorkload(), calculateWorkload()

Execution: When the student requests a workload analysis, PlannerService retrieves their academic tasks from StudentData. It uses the tasks' deadlines and estimated work to calculate the workload for the week. The results are returned to the GUI so the student can see how busy their week is.

F06 - Generate Weekly Plan

Related Use Case: UC06 - Generate Weekly Plan

Related Sequence Diagram: SD03 - Generate Weekly Plan

Classes involved:

- PlannerController - receives the planning request.
- PlannerService - coordinates weekly plan generation.
- WeeklyScheduler - creates the study schedule.
- SchedulingStrategy - determines how tasks should be prioritized.
- ConflictDetector - checks for scheduling conflicts.
- WeeklyPlan - stores the proposed schedule.

Important methods: generatePlan(), generate(), buildPlan(), hasConflict()

Execution: When the student requests a weekly plan, PlannerController sends the request to PlannerService. The service uses WeeklyScheduler to create study sessions based on the student's tasks, deadlines, and available time. The selected scheduling strategy determines how tasks are prioritized, while ConflictDetector checks for overlapping activities. The proposed plan is returned to the student for approval before it is saved.

F07 - Detect Schedule Conflicts

Related Use Case: UC07 - Detect Schedule Conflicts

Related Sequence Diagrams: SD03 - Generate Weekly Plan; SD04 - Reschedule Weekly Plan

Classes involved:

- WeeklyScheduler - requests conflict checks while creating a schedule.
- ConflictDetector - identifies overlapping activities.
- StudySession - represents a scheduled study period.
- StudentData - provides existing commitments and schedule information.

Important method: hasConflict()

Execution: While generating or rescheduling a weekly plan, WeeklyScheduler uses ConflictDetector to check whether a proposed study session overlaps with another scheduled activity. If a conflict is found, the scheduler must adjust the proposed schedule or report that a suitable time could not be found. This helps prevent study sessions from being placed during existing commitments.

F08 - Reschedule Weekly Plan

Related Use Case: UC08 - Reschedule Weekly Plan

Related Sequence Diagram: SD04 - Reschedule Weekly Plan

Classes involved:

- PlannerController - receives the rescheduling request.
- PlannerService - coordinates the changes.
- WeeklyScheduler - generates an updated schedule.
- SchedulingStrategy - determines how remaining tasks should be arranged.
- ConflictDetector - checks the updated schedule for conflicts.
- WeeklyPlan - represents the student's current and updated schedule.

Important methods: reschedulePlan(), reschedule(), buildPlan(), hasConflict()

Execution: When a student needs to change their weekly schedule, the request is sent to PlannerService through the controller. WeeklyScheduler uses the remaining tasks and available time to create an updated plan. ConflictDetector checks that the new sessions do not overlap with existing commitments. The updated plan is then shown to the student for approval, while completed sessions remain unchanged.

F09 - Track Habits and Progress

Related Use Case: UC09 - Track Habits and Progress

Related Sequence Diagram: SD05 - Track Habits and Progress

Classes involved:

- StudentFlowGUI - allows students to record habits and view progress.
- PlannerController - receives habit tracking requests.
- PlannerService - handles habit updates.
- Habit - stores information about a student's habits.
- StudentData - stores the habit records.

Important methods: recordHabit(), record(), save(), update()

Execution: When a student records a completed habit, the GUI sends the information to PlannerController. PlannerService updates the corresponding Habit object and saves the new information in StudentData. The GUI is then notified of the change and displays the updated progress.

F10 - Use AI Planner Assistant

Related Use Case: UC10 - Use AI Planner Assistant

Related Sequence Diagram: SD06 - Use AI Planner Assistant

Classes involved:

- StudentFlowGUI - allows students to enter questions or requests.

- PlannerController - forwards requests to the agent.
- PlanningAgent - interprets the student's request.
- AgentTools - allows the agent to access application information.
- PlannerService - provides access to the student's stored data.
- LLMClient - defines the interface for communicating with an AI model.
- OpenAIAdapter - connects the application to the external AI service.

Important methods: ask(), respond(), getTasks(), getData(), generateResponse()

Execution: The student enters a question or planning request through the GUI. PlannerController forwards it to PlanningAgent, which can use AgentTools to retrieve relevant information, such as upcoming tasks and deadlines. The agent communicates with the external AI model through LLMClient and OpenAIAdapter. The AI response is returned to the agent and then displayed to the student through the GUI.